
Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 2

Author(s)

Manasvi Bandi	bandim@mcmaster.ca	bandim
Nithya Majeti	majetin@mcmaster.ca	majetin
Mohit Gedela	gedelm1@mcmaster.ca	gedelm1
Lohitashwa Madhan	madhanl@mcmaster.ca	madhanl

GitHub URL

<https://github.com/MohitGedela/Team19-Catan.git>

<https://github.com/MohitGedela/Team19-Catan-V2.git>

March 11, 2026

1 Executive summary

This version of the simulator has expanded upon the previous version to create an interactive version of Catan where a human player is able to compete against other computer players. This has been accomplished through the creation of a command line interface that makes use of a regex-based parser to enable the human player to interact with the game environment. To ensure ease of use and flexibility of design, the Command pattern has been utilized to ensure that there is no coupling of data with its respective action.

A JSON-based visualizer has been implemented to provide a real-time visualization of the board environment to the human player. Furthermore, a "step forward" mode has been implemented to enable the human player to observe each move made by other players before proceeding with the next turn. The Robber feature has been fully implemented to enable the game to operate according to Catan rules. Throughout this design, consideration has been given to OOP principles and SOLID principles to ensure that there is a proper distribution of responsibilities across various classes and that abstract data is used to represent other data without directly referencing it.

The system design is documented through a UML class diagram covering all classes, relationships, and multiplicities. The system is validated through JUnit tests using boundary and partition strategies, covering parser correctness, resource management, building placement rules, and robber behaviour.

2 Requirements traceability

Req ID	Status	Implemented in	Design considerations
R2.1	{Implemented}	ParseCommand.java, HumanPlayer.java, Command.java, Roll.java, Go.java, List.java, Build.java	HumanPlayer.takeAction() takes input and gives it to ParseCommand.parse(), where input is compared against 6 regex patterns. If input is valid a command object is returned and invalid input returns null and re-prompts. Parsing is separated from execution (SRP). HumanPlayer calls command.requiresRoll() and command.endsTurn() with no instanceof checks (DIP).
R2.2	{Implemented}	Visualizer.java, Turn.java, Demonstrator.java.	Turn.execute() calls visualizer.refresh() after each player action. All visualization logic is contained in Visualizer, keeping it out of Player and Turn (SRP).

Continued on next page

Continued from previous page

Req ID	Status	Implemented in	Design considerations
R2.3	{Implemented}	Visualizer.java, visualize/state.json.	After each action Visualizer.refresh() overwrites state.json provided and ensures that everything is dynamically updated and checked for correctness.
R2.4	{Implemented}	ParseCommand.java, HumanPlayer.java, Command.java, Roll.java, Go.java, List.java, Build.java	The human types Go to end their turn. In between the computer turns, Simulator calls player.requiresGoPrompt() to decide whether to pause and ComputerPlayer returns true, HumanPlayer returns false. No instanceof in Simulator (DIP).
R2.5	{Implemented}	Robber.java, Turn.java, Player.java, Board.java	On a roll of 7, Robber.runRobber() discards half the cards of any player with >7, moves the robber to a random tile via Board.moveRobber(), then steals one card from a random adjacent player via Player.giveRandomResource()
R2.6	{Implemented}	Demonstrator.java	The main method that runs and demonstrates all the core game functionalities including board setup, player creation, the main game loop via Simulator.runGame() and each part is well commented referencing the requirements.

3 Getting ready for change

How many unit tests did you end up writing?

A total of 27 unit tests were written for the project. 11 tests use partition testing across the classes Catan, City, Settlement, Road, and ResourceHex. These tests verify different logical partitions such as valid inputs, null owners, resource production behaviour, and different configuration values for the game. 16 tests use boundary testing across the classes Board, Dice, Player, GameRules, and Turn. These tests focus on edge cases such as the lowest and highest valid intersection IDs, valid dice roll ranges (2–12), resource limits, and rule thresholds such as the 7-resource forced spend rule.

If you measured coverage: which tool did you use and what was your coverage?

For measuring code coverage, JUnit 5 was used in combination with the Eclipse IDE's built-in coverage tool, EcEmma. EcEmma is a free Java code coverage tool that integrates seamlessly with Eclipse and JUnit, providing detailed line, branch, and instruction level coverage metrics for each class under test. When the test suite is run using the "Coverage As JUnit Test" option in Eclipse, EcEmma automatically generates a coverage report that highlights

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Assignment1Test	89.5 %	3,018	354	3,372
src	89.5 %	3,018	354	3,372
(default package)	89.5 %	3,018	354	3,372
> GameRules.java	45.0 %	91	111	202
> Board.java	90.6 %	998	104	1,102
> Player.java	82.6 %	447	94	541
> Edge.java	47.4 %	18	20	38
> Simulator.java	89.1 %	82	10	92
> BoundaryTestSuite.java	0.0 %	0	3	3
> Catan.java	98.6 %	215	3	218
> DiceTest.java	96.0 %	72	3	75
> PartitionTestSuite.java	0.0 %	0	3	3
> DesertHex.java	77.8 %	7	2	9
> Production.java	98.7 %	75	1	76
> BoardTest.java	100.0 %	115	0	115
> Building.java	100.0 %	15	0	15
> CatanTest.java	100.0 %	42	0	42
> City.java	100.0 %	7	0	7
> CityTest.java	100.0 %	74	0	74
> Dice.java	100.0 %	37	0	37
> GameRulesTest.java	100.0 %	91	0	91
> HexBoardNum.java	100.0 %	9	0	9
> HexTerrain.java	100.0 %	15	0	15
> Intersection.java	100.0 %	29	0	29
> PlayerTest.java	100.0 %	159	0	159
> ResourceHex.java	100.0 %	13	0	13
> ResourceHexTest.java	100.0 %	35	0	35
> ResourceType.java	100.0 %	54	0	54
> Road.java	100.0 %	15	0	15
> RoadTest.java	100.0 %	43	0	43
> Settlement.java	100.0 %	7	0	7
> SettlementTest.java	100.0 %	55	0	55
> Turn.java	100.0 %	32	0	32
> TurnTest.java	100.0 %	166	0	166

Figure 1: Overall code coverage

which lines of code were executed during the test run and which lines were not reached. This allows for easy identification of any untested code paths or branches that may require additional test cases. The coverage is broken down per class, giving a clear picture of how thoroughly each component has been tested. The coverage results for each class; CatanTest, CityTest, RoadTest, SettlementTest, ResourceHexTest, Board, Dice, GameRules, Player, and Turn are all shown in the attached screenshot below, demonstrating the percentage of code covered by the partition test suite and highlighting any areas where coverage could potentially be improved with additional testing.

4 Changing and extending our design

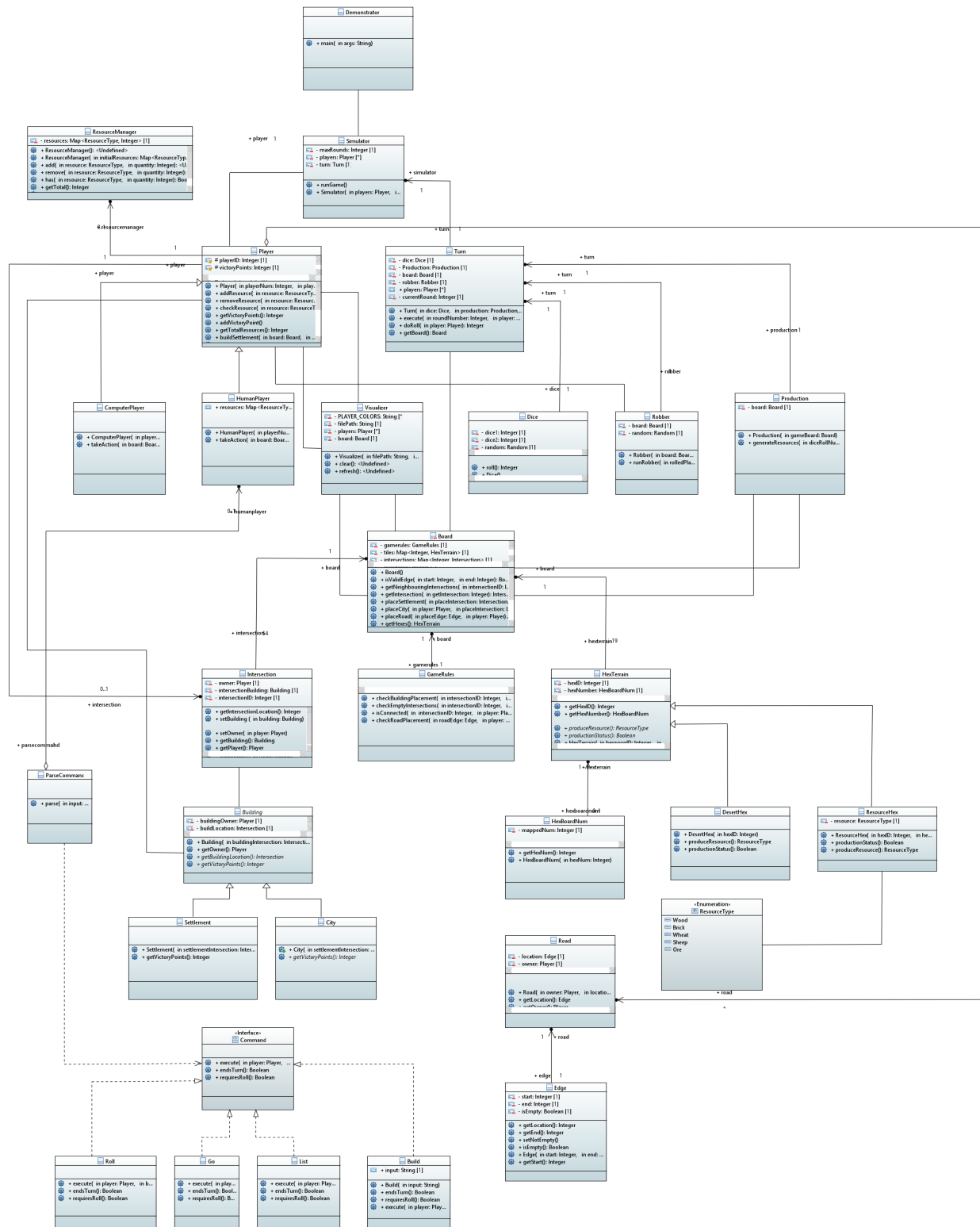


Figure 2: Updated UML Diagram

What were the main structural changes in your original design? Was it easy to localize the change and assess its impact compared to directly changing the code (without messing around with the design)?

The previous UML developed didn't cater to the requirements and specifications provided for the new extension of the Catan game. This has led to a few core structural changes within the design to allow for the addition of the new specifications. A main structural change introduced was initially the player class only reflected the behaviour of a computer agent and was hardcoded to exhibit the behaviour of a bot with random moves. However, with the introduction of a human player the structure had to change and the player class was made into an abstract class where the human player and the computer player inherit from the player class complying with the Open Closed principle and allows for easy future extensions. Another main structural change introduced to the design was for the code to reference only the building class rather than directly referencing settlement or city and this allowed for better Liskov Substitution Principle as the settlement or city can be swapped for Building and the code wouldn't break. By bringing these main structural changes before architecting and designing more changes within the UML has allowed for cleaner integration of all subsequent changes. Localizing these changes through the UML was significantly easier than it would have been working directly in code. The diagram made it immediately clear which classes depended on Player and Building, so the impact of making them abstract could be assessed before touching a single line of code.

Why was the automaton-based modelling useful and what benefits does it provide compared to the specification in the attached PDF? How did you translate the automaton-based model to program code?

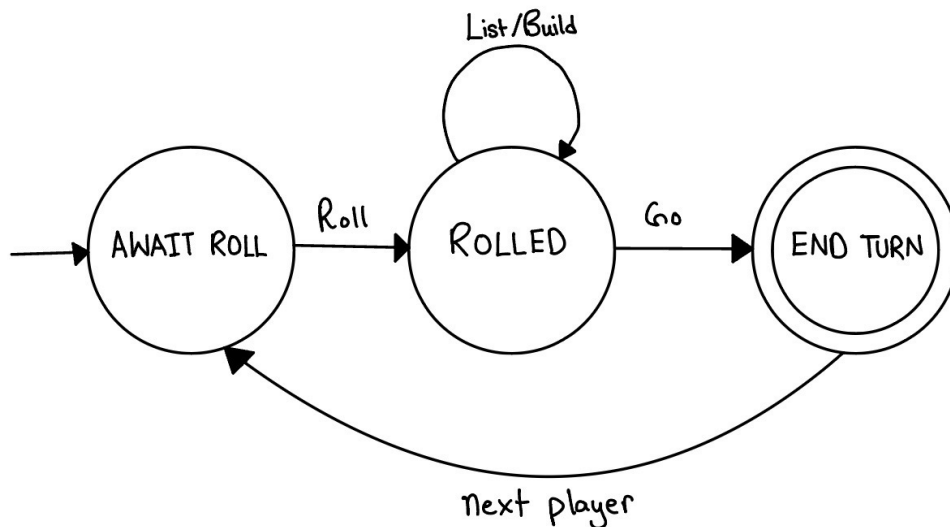


Figure 3: Automaton Model

The automaton made the action for the human player explicit and unambiguous. While the specification is good at stating the requirements, there is still room for interpretation. For example, it does not explicitly say that List and Build should be repeatable before ending a turn, or that Roll must come before anything else. The automaton makes these constraints obvious visually, so it is immediately clear if something is wrong (for example, typing Go before Roll. The automaton directly maps onto `HumanPlayer.takeAction()`. The AWAIT ROLL state is the entry point. The method only accepts a Roll command, since `command.requiresRoll()` is false only in this case. Once a Roll is entered, the player is in the ROLLED state, a while loop that accepts List and Build commands (`command.endsTurn()` is false) until Go is entered. When Go is entered, `command.endsTurn()` is true, and the loop is exited and we are in END TURN. The arc back to AWAIT ROLL of the next player is handled in `Simulator.runGame()`, which moves to the next player in the list.

5 Reflection on the coding process

Which parts of your system did break and how did you resolve those issues? **Visualizer**

When the Visualizer was first integrated, the JSON output was not matching the schema that the instructor-provided Python visualizer was expecting. The way that the internal state of the roads and the buildings was being represented was using different field names and structures than the visualizer was expecting. The field mappings for the roads were supposed to be {a, b, owner}, and the field mappings for the buildings were supposed to be {node, owner, type}. However, the initial implementation was using different key names and structures. This meant that the visualizer was crashing or displaying nothing. To solve the problem, it was necessary to revisit the `Visualizer.refresh()` method and remap the fields to match the schema that the visualizer was expecting. This included ensuring that the 'owner' field matched the player colour strings that the Python visualizer was expecting and that the 'type' field for the buildings was being output as the correct string ("settlement" or "city") rather than the Java enum.

Main Game Flow

When the human player was first added into the game loop, the flow was not working correctly because the `Simulator.runGame()` method was originally built around computer players that complete their turn in one call to their `takeAction()` method and immediately return. With the addition of the `HumanPlayer`, the flow was not correctly pausing between computer player turns; the simulation was running through all the computer player turns without giving the human player a chance to see the results of the computer players' turns.

Furthermore, the step forward mechanism was also not correctly integrated at first, which meant that the human player could enter their commands before the previous computer player's turn was completely resolved and the state updated correctly. The solution was to correctly add the `requiresGoPrompt()` check into the `Simulator.runGame()` method so that it blocks on the `nextLine()` method of the scanner until the human types "Go".

ParseCommand:

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Assignment1Test	6.0 %	247	3,854	4,101
src	6.0 %	247	3,854	4,101
(default package)	6.0 %	247	3,854	4,101
Board.java	0.0 %	0	1,175	1,175
ComputerPlayer.java	0.0 %	0	425	425
Player.java	0.0 %	0	332	332
Demonstrator.java	0.0 %	0	321	321
Visualizer.java	0.0 %	0	220	220
GameRules.java	0.0 %	0	202	202
HumanPlayer.java	0.0 %	0	196	196
ResourceManager.java	0.0 %	0	142	142
Simulator.java	0.0 %	0	116	116
Build.java	5.6 %	6	102	108
Robber.java	0.0 %	0	95	95
Turn.java	0.0 %	0	87	87
Production.java	0.0 %	0	79	79
List.java	0.0 %	0	61	61
ResourceType.java	0.0 %	0	54	54
Building.java	0.0 %	0	39	39
Edge.java	0.0 %	0	38	38
Dice.java	0.0 %	0	30	30
Intersection.java	0.0 %	0	29	29
Settlement.java	0.0 %	0	18	18
City.java	0.0 %	0	16	16
HexTerrain.java	0.0 %	0	15	15
Road.java	0.0 %	0	15	15
ResourceHex.java	0.0 %	0	13	13
DesertHex.java	0.0 %	0	9	9
HexBoardNum.java	0.0 %	0	9	9
Roll.java	27.3 %	3	8	11
Go.java	55.6 %	5	4	9
ParseCommand.java	92.9 %	52	4	56
ParseCommandBoundaryTest.java	100.0 %	73	0	73
ParseCommandPartitionTest.java	100.0 %	108	0	108

Figure 4: Coverage results for ParseCommand

The initial problem with these regex patterns was that they were not considering case insensitivity. As a result, some correct input such as 'roll' or 'ROLL' was causing null to be returned and prompting the user again. The solution was to add '(?i)' to each pattern.

The null return from these functions meant that a NullPointerException was being thrown in HumanPlayer's takeAction method because it was attempting to call command.endsTurn() without checking if command was null first.

What was the benefit of having unit tests at this stage?

Having unit tests at this stage was very helpful because the codebase changed significantly while fixing compilation errors and implementing the new parsing functionality. The unit tests acted as a safety net that quickly verified whether the program still behaved correctly after each modification. When changes were made to classes or method signatures, the tests immediately revealed if any existing functionality had broken.

The tests written for the ParseCommand class were especially useful while implementing the regular expression parser. They allowed the correctness of command recognition to be verified for multiple partitions and boundary cases, such as valid commands, invalid inputs, and edge cases like leading or trailing spaces. Instead of manually testing each command through the program interface, the unit tests automatically confirmed whether the parser

returned the correct command objects or null for invalid inputs.

Overall, the unit tests reduced debugging time, made it easier to refactor code with confidence, and ensured that new functionality did not unintentionally break existing behavior. Their coverage results can be viewed in Figure 4.

What were some of the challenges in these iterations (technical or conceptual)? What tools or integrated tool chains would help you in those steps? (You can specify a few abstract ideas, you don't have to name specific tools or techniques.)

The largest conceptual challenge was figuring out the player hierarchy prior to understanding all the requirements. The original Player class was completely computer-centric, and retrofitting it to be an abstract class with all the original computer player logic without breaking anything was a challenge. Careful consideration was required for each method to determine what was staying in the Player class, what was moving to the ComputerPlayer class, and what was becoming abstract.

Another challenge was incorporating the human player in a game system that was never intended to handle blocking inputs. The original simulation was intended to run from start to finish on its own, and a player was needed to introduce a wait for keyboard input. This required a significant shift in how the game system was managed in the Simulator class. The use of the Command pattern was also a challenge. Determining the proper level of methods for the interface was a challenge. If too many methods were in the Command class, implementations would be forced to implement methods they would never use. If too few methods were in the Command class, the calling code would be forced to understand too much about the specific class. Getting endsTurn and requiresRoll to be the correct two methods was a challenge in itself.

From a programming standpoint, getting the visualizer to stay in sync with the game was more difficult than expected. Since there are several classes that modify the board, figuring out the single right place to call refresh() without missing a call or calling it redundantly required tracing through the entire game flow.

Having a continuous integration environment that runs the entire test suite every time something changes would have caught those breaking changes right away instead of letting them slide until later. This is especially useful during a refactor of Player into an abstract class, where the effects throughout the codebase aren't obvious until something breaks. Having an AI-assisted code review assistant integrated in the workflow would have been very helpful. For example, it would have warned if a class is becoming too complex by doing too many things, if an instanceof test is due to a missing level of abstraction, or if a new dependency violates the intended architecture. Having these kinds of automated feedback during the initial phases would have helped me avoid some design mistakes from getting ingrained in the code.

Elaborate on the OO mechanisms and SOLID principles in your implementation. Are these any different from the ones used in the design? If they are: why?

The code openly uses all the principles of object-oriented programming, like inheritance,

polymorphism, abstraction, and encapsulation. A good example of inheritance is that Player is an abstract class with ComputerPlayer and HumanPlayer as subclasses. Another good example is that likeAction() and requiresGoPrompt() are called polymorphically by the Simulator without it knowing the type of the object it is calling. This is the same case with the Building, HexTerrain, and the Command classes. Encapsulation is maintained by the ResourceManager, which is responsible for all the resource handling, meaning that no other class can fiddle with a player's hand.

In regard to the principles of object-oriented programming, the single responsibility principle is clearly followed by the fact that each of the classes like ParseCommand, ResourceManager, GameRules, and Visualizer is a separate class. The open/closed principle is clearly followed by the fact that a new command can be added without changing the HumanPlayer class through the use of the interface for the Command classes. The Liskov substitution principle is clearly followed by the fact that the classes like Settlement, City, ComputerPlayer, and HumanPlayer can all be used anywhere their parent is needed without any problems. The interface segregation principle is clearly followed by the fact that the interface for the Command classes only has three methods, each for a specific purpose. Finally, the dependency inversion principle is clearly followed by the fact that the Simulator depends on the abstract Player, and HumanPlayer depends on the interface for the Command classes.

Difference from the design: The difference from the original design is that encapsulation was not fully followed by the fact that the fields of the Player class were not made private as shown in the UML, meaning that they could be accessed directly by the ComputerPlayer and HumanPlayer classes. Ideally, the subclasses should have made use of the public interface of the parent class, but it was pragmatically decided that a large set of getters would be needed, and that would be a lot of work.

6 Notes

- The provided mappings were not working, so the group decided to change the layout of our mappings to match the visualizer mappings.
- The group's unit tests were all done in JUnit, and also coverage reports.

7 Roles and responsibilities

The team members contributed equally to the deliverable.

- Manasvi Bandi: Boundary testing in Task 1. Testing for Parse Command class.
- Nithya Majeti: Programming the requirements in Task 3. Creating the automaton diagram for Task 2.
- Lohitashwa Madhan: Partition testing in Task 1. UML diagram for Task 2.
- Mohit Gedela: Programming the requirements in Task 3 and implementing the visualizer within the code base.